

Cryptocurrency Price Analysis





UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II

Statistical Learning and Data Analysis

Data Science

Prof. Roberta Siciliano

Dataset : [Cryptocurrency Price Analysis Dataset](#)

Project Code: https://colab.research.google.com/drive/1VUYCAbnx8AE-rIN6ZgPZyAUmkPw_hHXy?usp=sharing



D03000157 Sonal Sunit Deshmukh

• Cryptocurrency Price Analysis •

WHAT IS CRYPTOCURRENCY ?

A cryptocurrency is a digital or virtual currency that uses cryptography for security and operates on a decentralized blockchain. Not controlled by any central authority, it allows for peer-to-peer transactions without intermediaries, facilitating quicker and more secure financial exchanges.



HOW CRYPTOCURRENCY WORKS?

Cryptocurrencies function through a process where transactions are recorded on a blockchain and verified by a network of computers (nodes). Mechanisms like Proof of Work (PoW) or Proof of Stake (PoS) ensure transaction integrity, while miners or validators are rewarded for their contributions to network security.



WHY CRYPTOCURRENCY ANALYSIS ?

cryptocurrency is a digital or virtual currency that uses cryptography for security and operates on a decentralized blockchain. Not controlled by any central authority, it allows for peer-to-peer transactions without intermediaries, facilitating quicker and more secure financial exchanges.



• Cryptocurrency Price Analysis •

OBJECTIVE OF THE ANALYSIS

The primary objectives of this analysis include:

- Understanding price distribution, performing correlation analysis among cryptocurrencies
- Assessing the influence of market capitalization on price stability.
- It aims to build predictive models for future price movements
- Identify trading strategies based on statistical insights gained from the data.
- Test whether cryptocurrency price changes follow a normal distribution.
- Perform hypothesis testing on the relationship between price and trading volume. Analyze whether market capitalization significantly affects price stability.
- Discuss how data balancing and feature engineering impact model accuracy.



• Cryptocurrency Price Analysis •



EXPLORATORY DATA ANALYSIS

- Data Understanding
- Data Visualization
- Missing Values
- Data Integration



DATA PROCESSING

- Handling Outliers
- Feature Engineering
- Training-Testing Data
- Data Balancing
- Under-Sampling
- Hypothesis Testing



MODEL TRAINING

- Logistic Regression
- Decision Tree
- Random Forest



MODEL COMPARISON

- Confusion Matrix
- Best Performance
- Comparison



CONCLUSION

- Result
- Key Takeaways
- Future Use
- Research Directions

• Cryptocurrency Price Analysis •

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import Ridge, Lasso, LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import mean_squared_error, r2_score, confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import RandomUnderSampler
from scipy import stats

# Load the dataset
df = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/crypto_combine.csv")
```

OVERVIEW OF DATASET

The Cryptocurrency Price Analysis Dataset encompasses historical price data for multiple cryptocurrencies, such as Bitcoin, Ethereum, and Ripple, from January 1, 2018, to May 31, 2023. It contains 7,899 rows of data detailing daily price movements, which serve as a fundamental resource for predictive modeling and market trend analysis.

IMPORT LIBRARIES

- A **Python library** is a collection of related modules. It contains bundles of code that can be used repeatedly in different programs.
- Key libraries utilized in this analysis include **Pandas** for data manipulation, **NumPy** for numerical operations, **Matplotlib** and **Seaborn** for data visualization, **Scikit-learn** for machine learning.
- These tools form a powerful ensemble that facilitates effective data handling, modeling, and visualization.

• Cryptocurrency Price Analysis •

```
# Get summary statistics of the dataset
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7899 entries, 0 to 7898
Data columns (total 6 columns):
#   Column  Non-Null Count  Dtype  
---  -
0   Crypto  7899 non-null      object  
1   Date    7899 non-null      object  
2   Open    7899 non-null      float64 
3   High    7899 non-null      float64 
4   Low     7899 non-null      float64 
5   Close   7899 non-null      float64 
dtypes: float64(4), object(2)
```

```
df.describe()
```

	Open	High	Low	Close
count	7899.000000	7899.000000	7899.000000	7899.000000
mean	5479.620176	5623.947143	5324.149024	5480.870398
std	12010.579332	12325.039614	11665.236339	12011.567525
min	0.140000	0.150000	0.120000	0.140000
25%	12.720000	13.410000	12.405000	12.680000
50%	170.910000	176.240000	163.320000	170.890000
75%	3879.000000	3962.000000	3757.000000	3878.935000
max	67802.000000	68925.000000	66112.000000	67802.000000

UNDERSTANDING DATASET

The dataset contains 7,899 rows and 6 columns.

- **Date:** The date of the recorded cryptocurrency data.
- **Cryptocurrency Name:** The name of the cryptocurrency (e.g., Bitcoin, Ethereum, etc.).
- **Open Price:** The price of the cryptocurrency at the start of the trading period.
- **Close Price:** The price of the cryptocurrency at the end of the trading period.
- **High Price:** The highest price reached by the cryptocurrency during the trading period.
- **Low Price:** The lowest price reached by the cryptocurrency during the trading period.

INSIGHT OF DATASET

Data types:

- 4 columns are of type float64 (numerical price data).
- 2 columns are of type object (text-based data: Crypto name and Date).

We used df.info(), df.describe(), df.columns, df.head(), df.tail(), df.shape to get insights of our data set.

```
# Display the first few rows of the dataset
df.head()
```

	Crypto	Date	Open	High	Low	Close
0	BTC	12/31/19	7254.0	7309.0	7132.0	7171.0
1	BTC	12/30/19	7402.0	7430.0	7217.0	7254.0
2	BTC	12/29/19	7334.0	7529.0	7295.0	7402.0
3	BTC	12/28/19	7235.0	7359.0	7235.0	7334.0
4	BTC	12/27/19	7208.0	7267.0	7087.0	7235.0

```
# Display the last few rows of the dataset
df.tail()
```

	Crypto	Date	Open	High	Low	Close
7894	XRP	1/5/19	0.36	0.36	0.35	0.36
7895	XRP	1/4/19	0.36	0.36	0.35	0.36
7896	XRP	1/3/19	0.37	0.38	0.35	0.36
7897	XRP	1/2/19	0.36	0.37	0.36	0.37
7898	XRP	1/1/19	0.35	0.36	0.34	0.36

```
df.shape
```

```
(7899, 6)
```



EXPLORETRLY DATA ANALYSIS

EDA involves summarizing the dataset's main characteristics through visualizations and statistical methods. It aids in uncovering data patterns, informing feature selection, and preparing for further modeling by highlighting trends and potential anomalies.

```
# Data Classification - Identifying numerical and categorical features
numeric_cols = df.select_dtypes(include=['int64', 'float64']).columns
categorical_cols = df.select_dtypes(include=['object']).columns

print("\nNumeric Columns:", numeric_cols)
print("Categorical Columns:", categorical_cols)

Numeric Columns: Index(['Open', 'High', 'Low', 'Close'], dtype='object')
Categorical Columns: Index(['Crypto'], dtype='object')
```

. DATA CLASSIFICATION

The dataset comprises 6 columns: Date, Cryptocurrency Name, Open Price, Close Price, High Price, and Low Price. We classified the columns to separate numerical and categorical features, as they require different preprocessing and modeling techniques. Numerical data can be scaled or normalized, while categorical data may need encoding (e.g., one-hot encoding) before feeding into machine learning models.

MISSING VALUE HANDLING

Missing values can severely affect model performance, necessitating proper imputation techniques such as mean, median, or mode replacement.

Check for Missing Values:

`df.isnull().sum()` calculates the count of missing values in each column. It prints the count of missing values before handling.

Conditionally Handle Missing Values:

If the total missing values (`missing_values.sum()== zero`): It prints a message saying no missing values are detected and proceeds further.

Otherwise:

For numerical columns (`numeric_cols`), missing values are filled using the median of each column. It then prints the count of missing values after handling.

Handling Missing Values

```
missing_values = df.isnull().sum()
print("Missing Values Before Handling:")
print(missing_values)

if missing_values.sum() == 0:
    print("\n No missing values detected. Proceeding with outlier detection and feature engineering.\n")
else:
    # Fill missing values with median for numerical columns
    df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].median())
    print("\n Missing Values After Handling:")
    print(df.isnull().sum())
```

Missing Values Before Handling:

```
Crypto    0
Date      0
Open      0
High      0
Low       0
Close     0
dtype: int64
```

No missing values detected. Proceeding with outlier detection and feature engineering.

```
# Data Integration
# Assuming integration involves aggregating data by date across all cryptocurrencies
# Calculate daily mean prices and total volume
daily_data = df.groupby('Date').agg({
    'Open': 'mean',
    'Close': 'mean',
    'High': 'mean',
    'Low': 'mean'
}).reset_index()

# Display the aggregated daily_data DataFrame
print("Aggregated Daily Data:")
print(daily_data.head())
```

Aggregated Daily Data:

	Date	Open	Close	High	Low
0	2018-01-01	3743.1750	3629.9350	3761.9625	3451.1975
1	2018-01-02	3630.1025	3972.7025	4091.0300	3484.9875
2	2018-01-03	3972.1975	4060.1950	4151.5775	3920.6750
3	2018-01-04	4059.9925	4026.4100	4158.5650	3839.3375
4	2018-01-05	4026.4075	4507.2750	4545.4600	3988.7875

DATA INTEGRATION

Data Integration is the process of combining data from multiple sources into a unified dataset. It ensures consistency, completeness, and accuracy in data analysis.



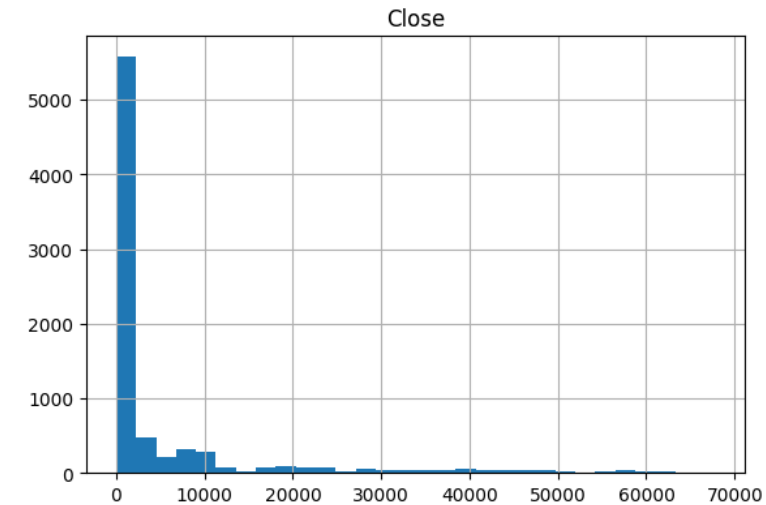
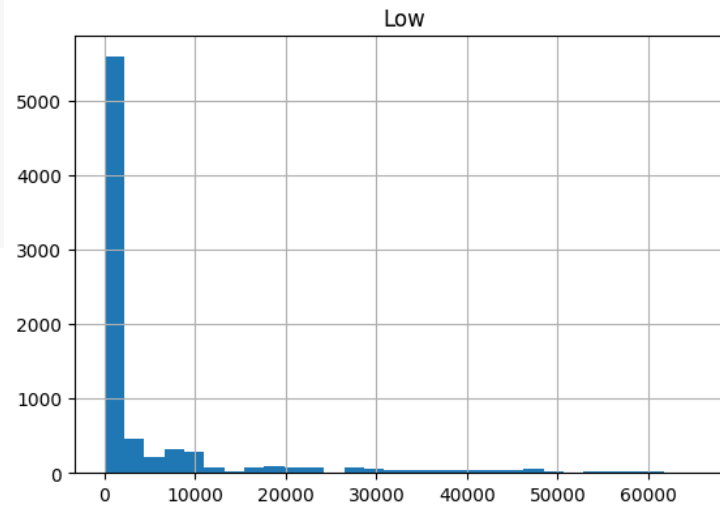
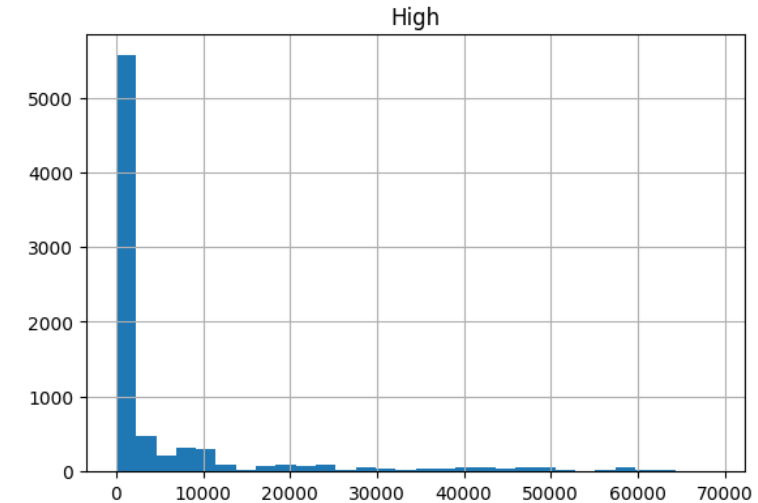
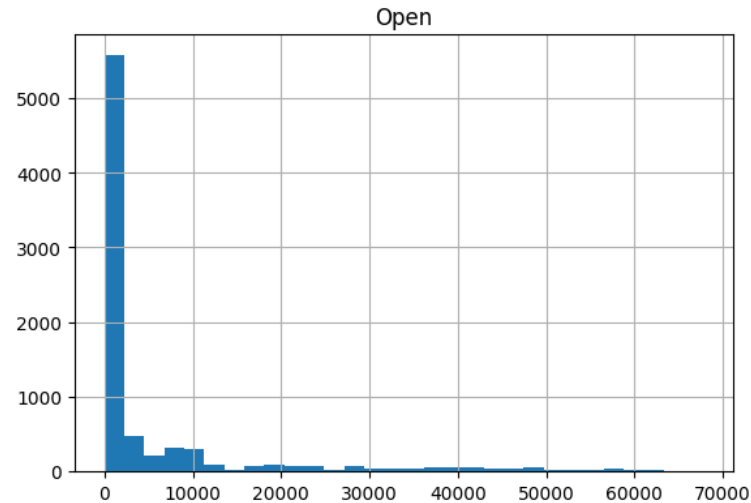
• Cryptocurrency Price Analysis •

DATA VISUALIZATION

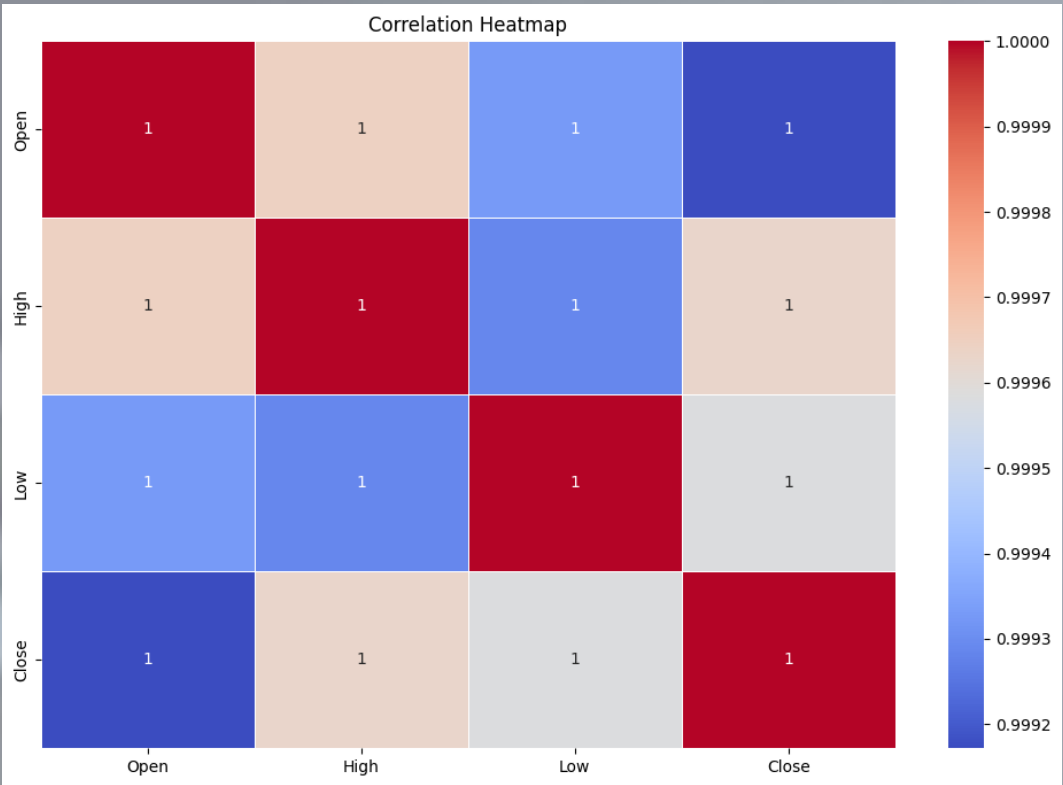
Data visualization refers to the graphical representation of data to help understand its underlying patterns, trends, and relationships. It involves creating charts, graphs, and plots that make it easier to identify key insights, detect anomalies, and summarize the distribution and relationships in the data.

```
# Data Visualization
# Histogram of numerical features
df[numeric_cols].hist(bins=30, figsize=(15, 10))
plt.suptitle('Histograms of Numerical Features')
plt.show()
```

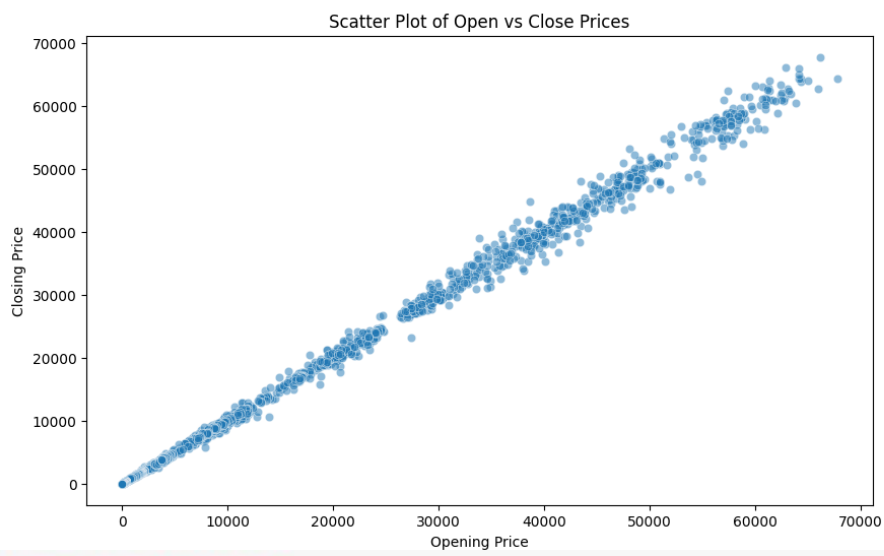
Histograms of Numerical Features



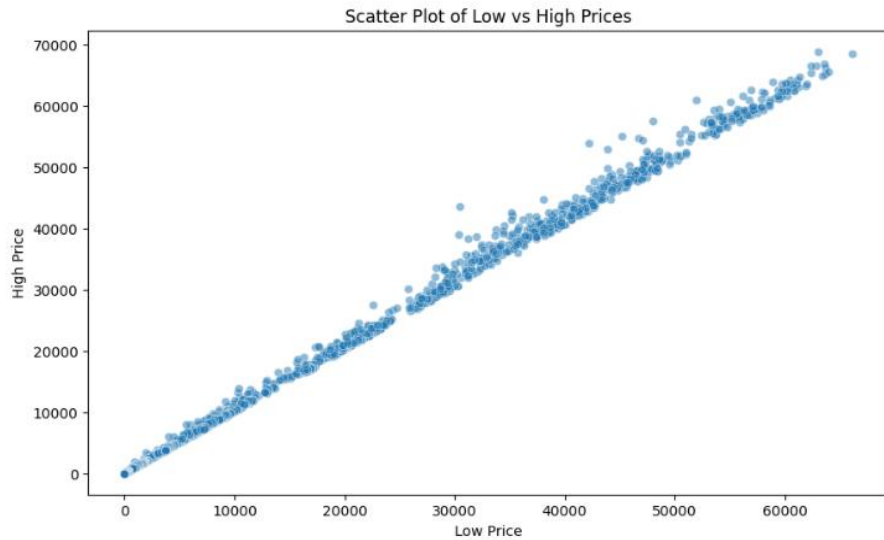
```
# Correlation heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(df.select_dtypes(include=['int64', 'float64']).corr(), annot=True, cmap='coolwarm', linewidths=0.5)
plt.title('Correlation Heatmap')
plt.show()
```



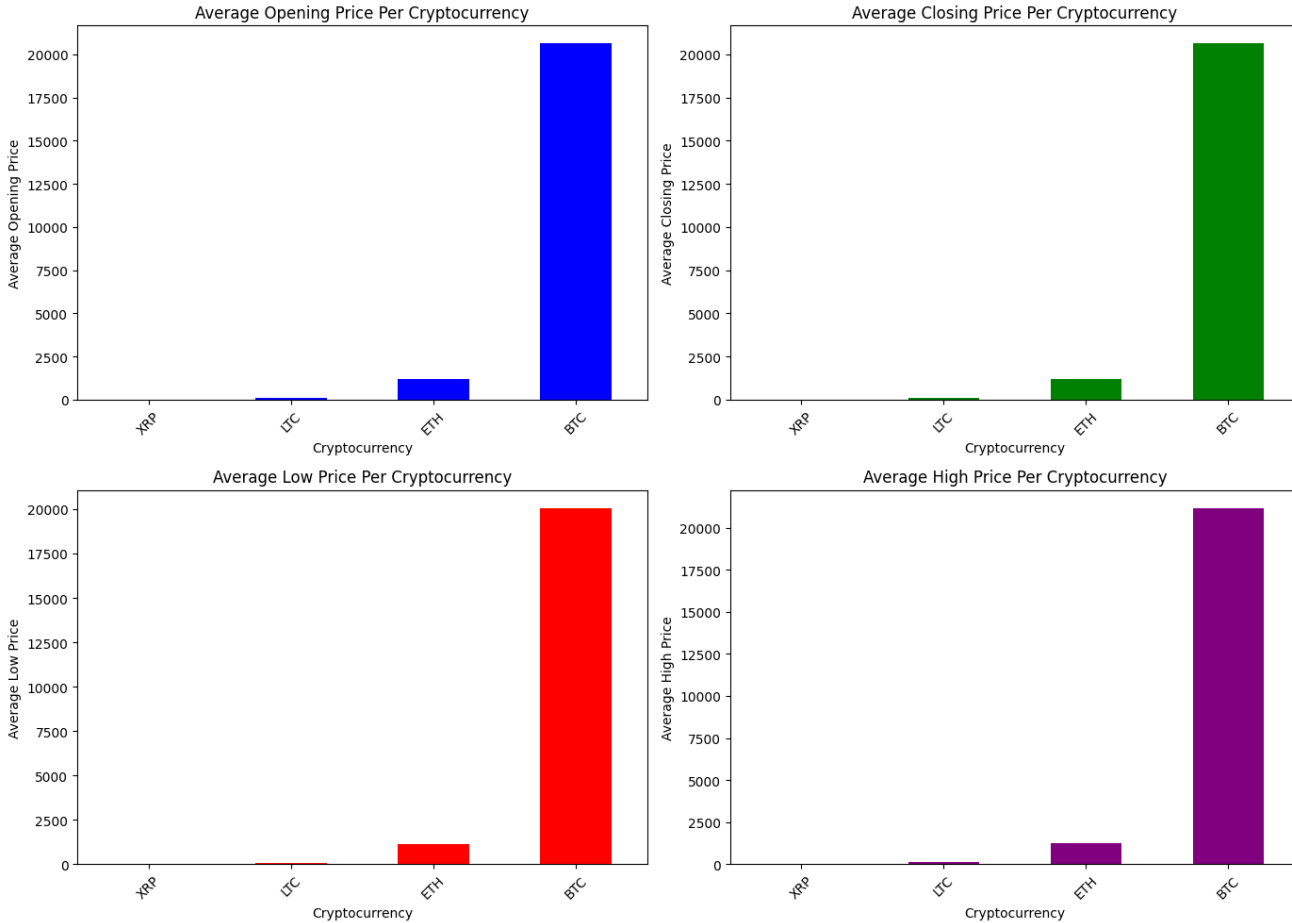
```
# Scatter Plot for Opening vs Closing Price
plt.figure(figsize=(10, 6))
sns.scatterplot(x=df['Open'], y=df['Close'], alpha=0.5)
plt.title('Scatter Plot of Open vs Close Prices')
plt.xlabel('Opening Price')
plt.ylabel('Closing Price')
plt.show()
```



```
# Scatter Plot for Low vs High Price
plt.figure(figsize=(10, 6))
sns.scatterplot(x=df['Low'], y=df['High'], alpha=0.5)
plt.title('Scatter Plot of Low vs High Prices')
plt.xlabel('Low Price')
plt.ylabel('High Price')
plt.show()
```



• Cryptocurrency Price Analysis •



```
# Bar Plot for Average opening, closing, high and low Price Per Cryptocurrency

# Define price types and colors
price_types = {'Open': 'blue', 'Close': 'green', 'Low': 'red', 'High': 'purple'}
titles = {'Open': 'Opening', 'Close': 'Closing', 'Low': 'Low', 'High': 'High'}

# Create subplots
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.flatten()

for i, (price_type, color) in enumerate(price_types.items()):
    df.groupby('Crypto')[price_type].mean().sort_values().plot(kind='bar', color=color, ax=axes[i])
    axes[i].set_title(f'Average {titles[price_type]} Price Per Cryptocurrency')
    axes[i].set_xlabel('Cryptocurrency')
    axes[i].set_ylabel(f'Average {titles[price_type]} Price')
    axes[i].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()
```

• Cryptocurrency Price Analysis •

```
# Create subplots

# Line Plot for HIGH Prices, Low Prices, Opening and Closing Prices Over Time

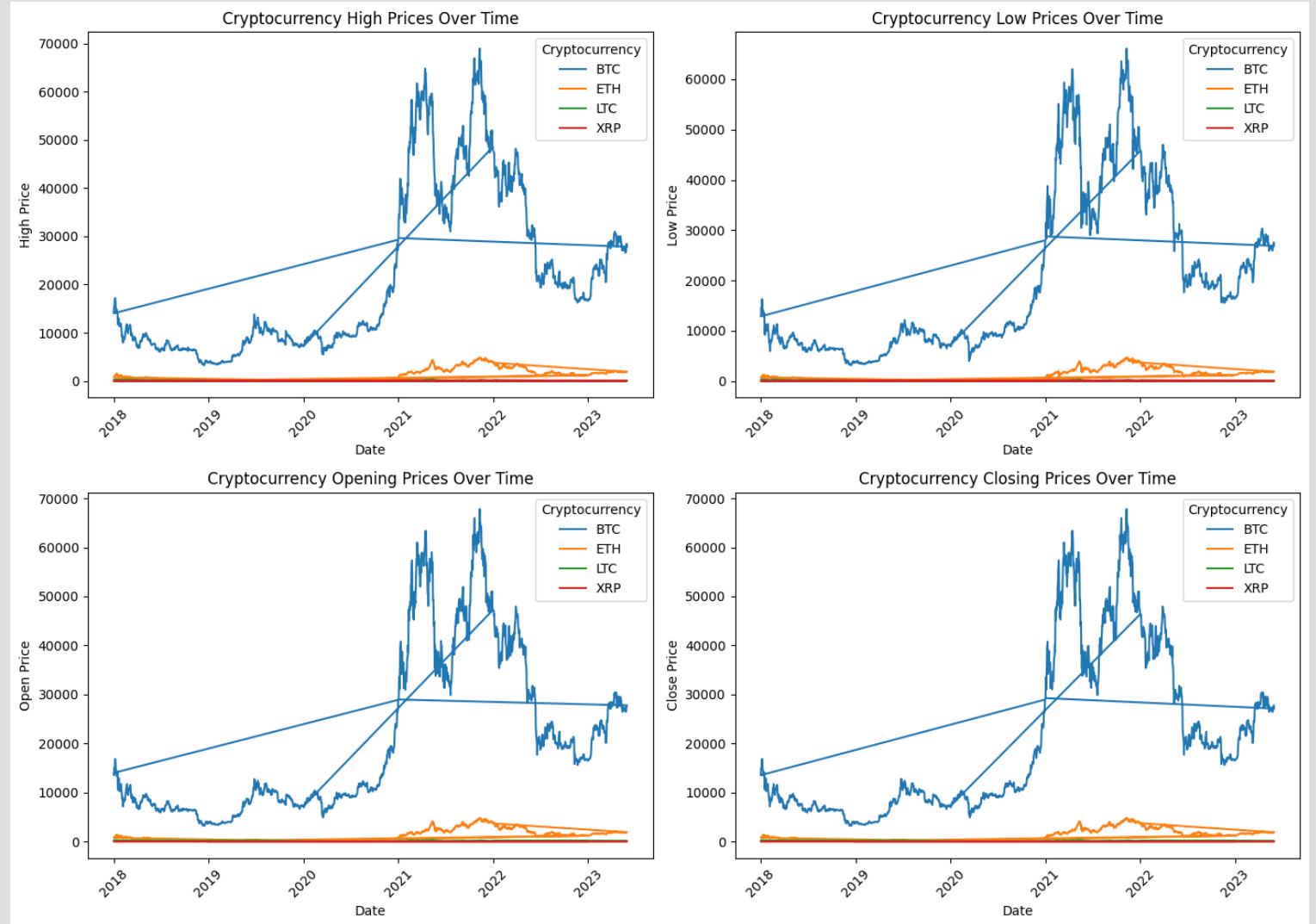
# Define price types and titles
price_types = ['High', 'Low', 'Open', 'Close']
titles = ['High Prices', 'Low Prices', 'Opening Prices', 'Closing Prices']

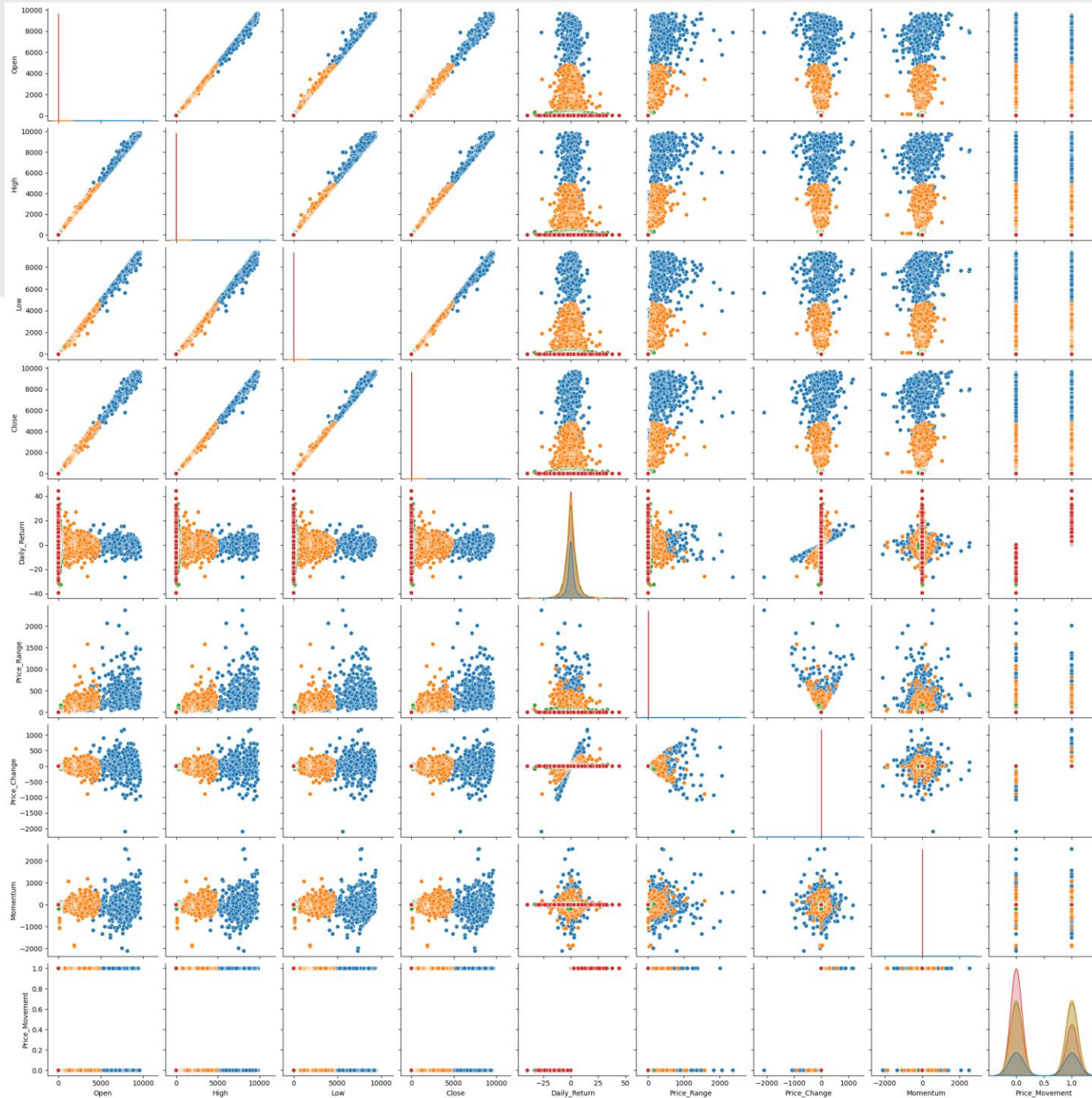
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.flatten()

for i, price_type in enumerate(price_types):
    for crypto in df['Crypto'].unique():
        subset = df[df['Crypto'] == crypto]
        axes[i].plot(subset['Date'], subset[price_type], label=crypto)

    axes[i].set_title(f'Cryptocurrency {titles[i]} Over Time')
    axes[i].set_xlabel('Date')
    axes[i].set_ylabel(f'{price_type} Price')
    axes[i].legend(title='Cryptocurrency')
    axes[i].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()
```





```
# Pairplot for all numerical features
sns.pairplot(df, hue='Crypto')
plt.show()
```

PAIR PLOT

A pairplot (from Seaborn library) creates scatter plots between all numerical feature pairs in a dataset.

It is useful for visualizing the relationships between multiple numerical variables and detecting trends or correlations.

From this heatmap we know that,

Highly Correlated Features: Features like opening, closing, high, and low prices are likely highly correlated, making some of them redundant for modeling.

Outliers: The presence of some extreme points suggests possible outliers (e.g., BTC price spikes).

Feature Scaling Needed: Since BTC prices are much higher than others, normalization or log transformation might be needed for better modeling.

Outlier Detection & Removal

```
# Display the shape of the dataset before outlier removal
print("\nDataset shape before outlier removal:", df.shape)
```

```
Q1 = df[numeric_cols].quantile(0.25)
Q3 = df[numeric_cols].quantile(0.75)
IQR = Q3 - Q1
outlier_condition = (
    (df[numeric_cols] < (Q1 - 1.5 * IQR)) | (df[numeric_cols] > (Q3 + 1.5 * IQR))
).any(axis=1)
```

```
print("\nNumber of outliers detected:", outlier_condition.sum())
df = df[~outlier_condition]
print("\nDataset shape after outlier removal:", df.shape)
```

Dataset shape before outlier removal: (7899, 6)

Number of outliers detected: 1233

Dataset shape after outlier removal: (6666, 6)

OUTLIER DETECTION

Outliers are data points that significantly differ from other observations. They can skew results and affect the performance of models. Handling outliers involves identifying and either removing or transforming them to minimize their impact.

1. Calculate Quartiles and IQR:

First Quartile (Q1): The 25th percentile of the data.

Third Quartile (Q3): The 75th percentile. Interquartile

Range (IQR): The difference between Q3 and Q1, representing the middle 50% of the data.

2. Define Outlier Boundaries:

Lower Bound: $Q1 - 1.5 * IQR$

Upper Bound: $Q3 + 1.5 * IQR$

3. Identify and Remove Outliers:

Data points falling below the lower bound or above the upper bound are considered outliers. These outliers are then removed from the dataset. This method is effective for datasets with a roughly normal distribution. However, for skewed distributions or datasets with varying variability, alternative methods like the Z-score or domain specific approaches might be more appropriate.

By applying the IQR method, we ensure that extreme values don't skew analysis, leading to more reliable and accurate results.

FEATURE ENGINEERING

Feature creation involves generating new variables (features) from existing data to improve model performance. This can include combining features, creating interaction terms, or applying domain-specific transformations.

Daily Return Percentage: Measures the percentage change between the opening and closing price of the day. It can be useful for volatility and trend analysis.

Price Range: Calculates the difference between the highest and lowest price to assess daily volatility. It can be useful for capturing market volatility.

Price Change: Finds the absolute price movement between the opening and closing price. It helps understand the direction and magnitude of price movement.

Momentum: Tracks the closing price difference over 3 days to identify trends. It helps in identifying short-term trends.

Price Movement: A binary indicator (1 = Up, 0 = Down) showing if the price increased or decreased. It can be useful for classification models predicting market movement.

```
# Ensure Date is in datetime format
df['Date'] = pd.to_datetime(df['Date'])

# Daily Return Percentage
df['Daily_Return'] = ((df['Close'] - df['Open']) / df['Open']) * 100

# Price Range
df['Price_Range'] = df['High'] - df['Low']

# Price Change
df['Price_Change'] = df['Close'] - df['Open']

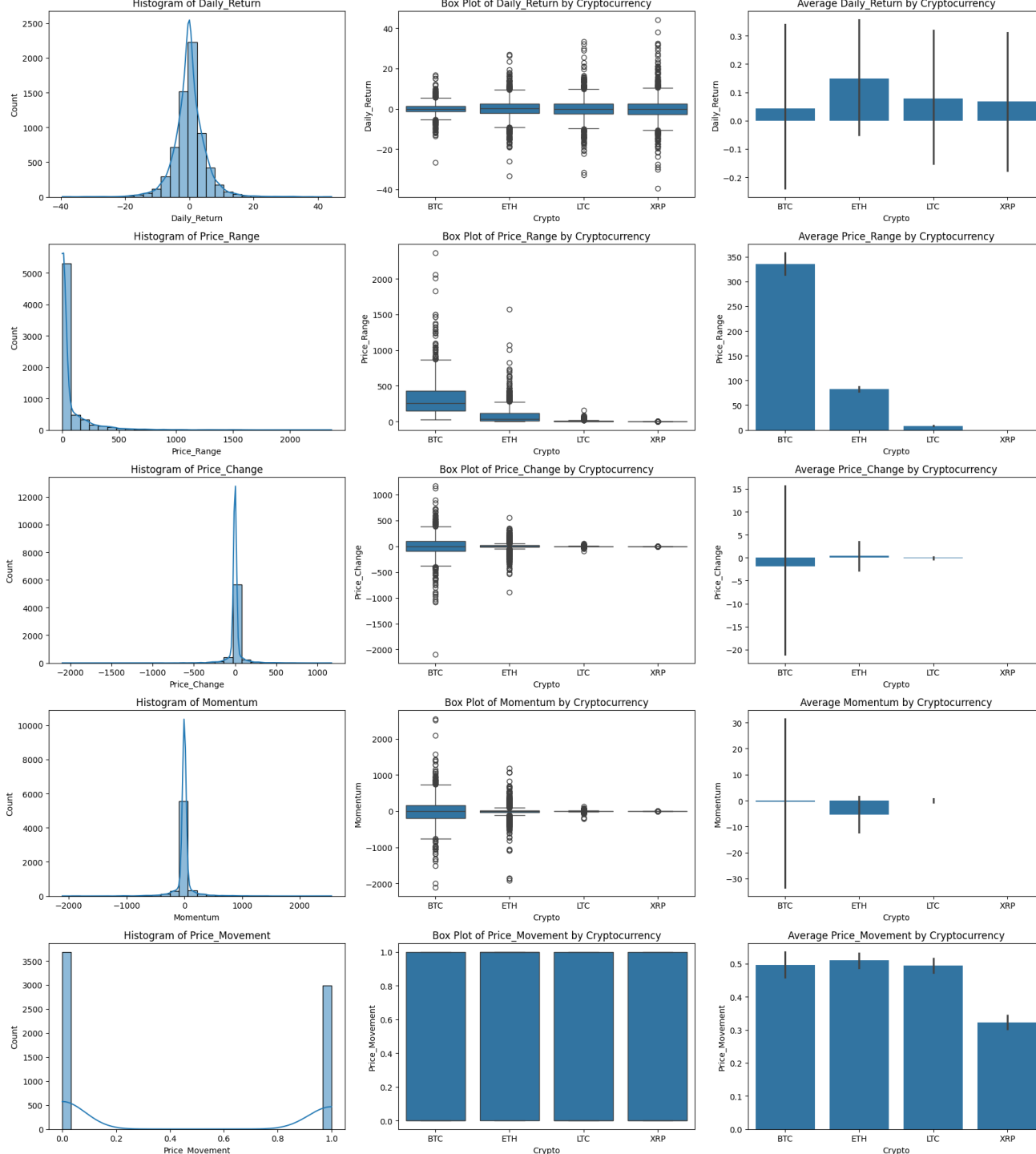
# Momentum (Close Price Difference over 3 days)
df['Momentum'] = df.groupby('Crypto')['Close'].transform(lambda x: x.diff(periods=3))

# Price Movements
# 1 for upward movement, 0 for downward
df['Price_Movement'] = np.where(df['Daily_Return'] > 0, 1, 0)

# Print the first few rows with new features
print(df[['Crypto', 'Date', 'Daily_Return', 'Price_Range', 'Price_Change', 'Momentum', 'Price_Movement']].head())
```

	Crypto	Date	Daily_Return	Price_Range	Price_Change	Momentum	\
0	BTC	2019-12-31	-1.144196	177.0	-83.0	NaN	
1	BTC	2019-12-30	-1.999460	213.0	-148.0	NaN	
2	BTC	2019-12-29	0.927188	234.0	68.0	NaN	
3	BTC	2019-12-28	1.368348	124.0	99.0	163.0	
4	BTC	2019-12-27	0.374584	180.0	27.0	-19.0	

	Price_Movement
0	0
1	0
2	1
3	1
4	1



Visualization

```
features = ['Daily_Return', 'Price_Range', 'Price_Change', 'Momentum', 'Price_Movement']
fig, axes = plt.subplots(len(features), 3, figsize=(18, 20))
```

```
for i, feature in enumerate(features):
```

Histogram

```
sns.histplot(df[feature], bins=30, kde=True, ax=axes[i, 0])
axes[i, 0].set_title(f'Histogram of {feature}')
```

Box Plot

```
sns.boxplot(x='Crypto', y=feature, data=df, ax=axes[i, 1])
axes[i, 1].set_title(f'Box Plot of {feature} by Cryptocurrency')
```

Bar Graph

```
sns.barplot(x='Crypto', y=feature, data=df, ax=axes[i, 2])
axes[i, 2].set_title(f'Average {feature} by Cryptocurrency')
```

```
plt.tight_layout()
plt.show()
```

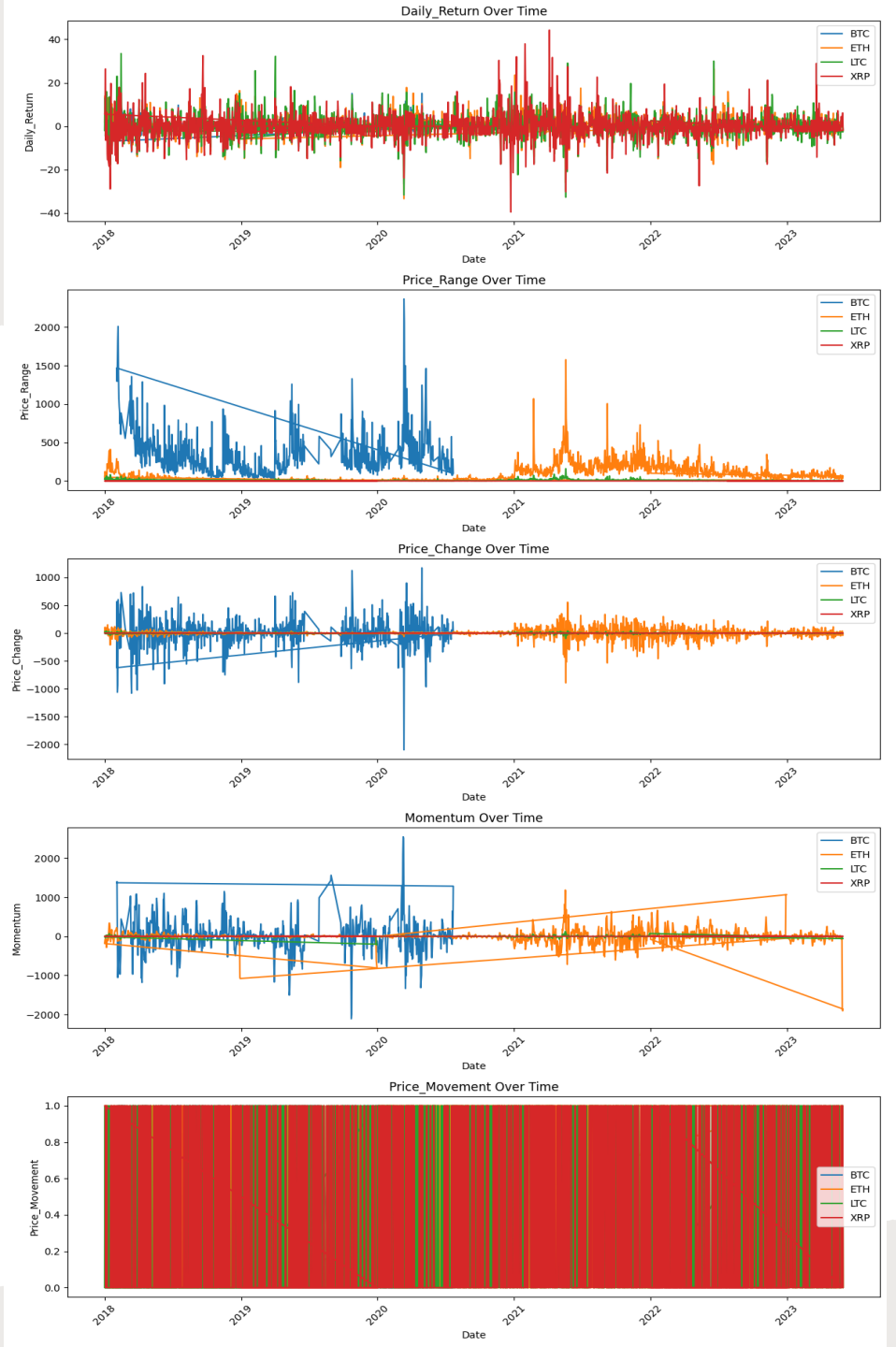
```
# Line Plot for All Features
```

```
features = ['Daily_Return', 'Price_Range', 'Price_Change', 'Momentum', 'Price_Movement']
fig, axes = plt.subplots(len(features), 1, figsize=(12, 20))
```

```
for i, feature in enumerate(features):
    for crypto in df['Crypto'].unique():
        subset = df[df['Crypto'] == crypto]
        axes[i].plot(subset['Date'], subset[feature], label=crypto)
    axes[i].set_title(f'{feature} Over Time')
    axes[i].set_xlabel('Date')
    axes[i].set_ylabel(feature)
    axes[i].legend()
    axes[i].tick_params(axis='x', rotation=45)
```

```
plt.tight_layout()
```

```
plt.show()
```



TESTING-TRAINING OF DATA

Training data is used to build and train a model, while testing data is used to evaluate its performance. This separation helps ensure the model generalizes well to unseen data.

Why Do We Train and Test Data?

In machine learning, we train and test data to evaluate how well a model generalizes to new, unseen data.

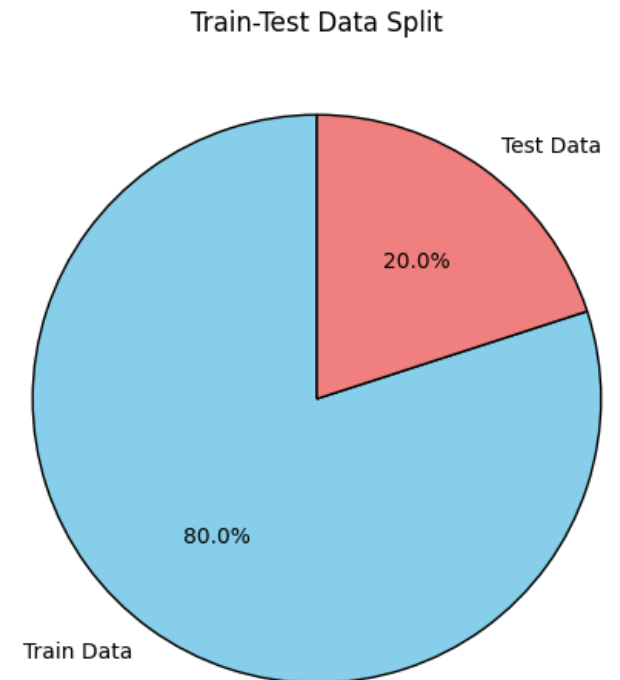
- **Prevent Overfitting** – If we train on all the data, the model may memorize the dataset instead of learning general patterns.
- **Evaluate Model Performance** – The test data acts as unseen data to check if the model performs well beyond the training set.
- **Hyperparameter Tuning** – Helps in fine-tuning the model (e.g., adjusting regularization parameters).
- **Measuring Accuracy** – Enables checking metrics like accuracy, precision, recall, and F1-score.

```
# Train-Test Split
X = df[numeric_cols].dropna()
y = np.where(df['Daily_Return'] > 0, 1, 0)
# Classification Target: 1 for upward movement, 0 for downward
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Count the number of samples in train and test sets
train_size = len(y_train)
test_size = len(y_test)

# Labels and sizes for the pie chart
labels = ['Train Data', 'Test Data']
sizes = [train_size, test_size]
colors = ['skyblue', 'lightcoral']

# Create a pie chart
plt.figure(figsize=(6, 6))
plt.pie(sizes, labels=labels, autopct='%1.1f%%', colors=colors, startangle=90, wedgeprops={'edgecolor': 'black'})
plt.title('Train-Test Data Split')
plt.show()
```



DATA BALANCING

Data balancing addresses imbalances in the dataset, such as unequal class distributions. Techniques like SMOTE (Synthetic Minority Over-sampling Technique) and under-sampling are used to balance the data.

What is SMOTE?

SMOTE (Synthetic Minority Over-sampling Technique) is a technique used to handle imbalanced datasets by generating synthetic samples for the minority class.

The **Synthetic Minority Over-Sampling Technique (SMOTE)** is a powerful method used to handle class imbalance in datasets. SMOTE handles this issue by generating samples of minority classes to make the class distribution balanced.

Why Do We Use SMOTE?

In classification problems, if one class has significantly fewer samples than the other(s), the model may become biased toward the majority class, leading to poor performance in predicting the minority class.

SMOTE helps by:

- **Balancing the dataset** by creating synthetic data points for the minority class.
- **Preventing overfitting** caused by simply duplicating existing data points.
- **Improving model performance** by allowing better learning from both classes.

```
# Data Balancing
# Print class distribution before balancing
print("\nClass distribution before balancing:")
print(pd.Series(y_train).value_counts())

# Visualize class distribution before balancing
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
pd.Series(y_train).value_counts().plot(kind='bar', color=['blue', 'red'])
plt.title("\nClass Distribution Before Balancing")
plt.xlabel("Class")
plt.ylabel("Count")

# SMOTE Balancing
smote = SMOTE(random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)

# Print class distribution after SMOTE
print("\nClass distribution after SMOTE:")
print(pd.Series(y_train_smote).value_counts())

# Visualize class distribution after SMOTE
plt.subplot(1, 2, 2)
pd.Series(y_train_smote).value_counts().plot(kind='bar', color=['blue', 'red'])
plt.title("\nClass Distribution After SMOTE")
plt.xlabel("Class")
plt.ylabel("Count")
plt.show()
```

```
Class distribution before balancing:
0    2958
1    2374
Name: count, dtype: int64
```

```
Class distribution after SMOTE:
1    2958
0    2958
Name: count, dtype: int64
```

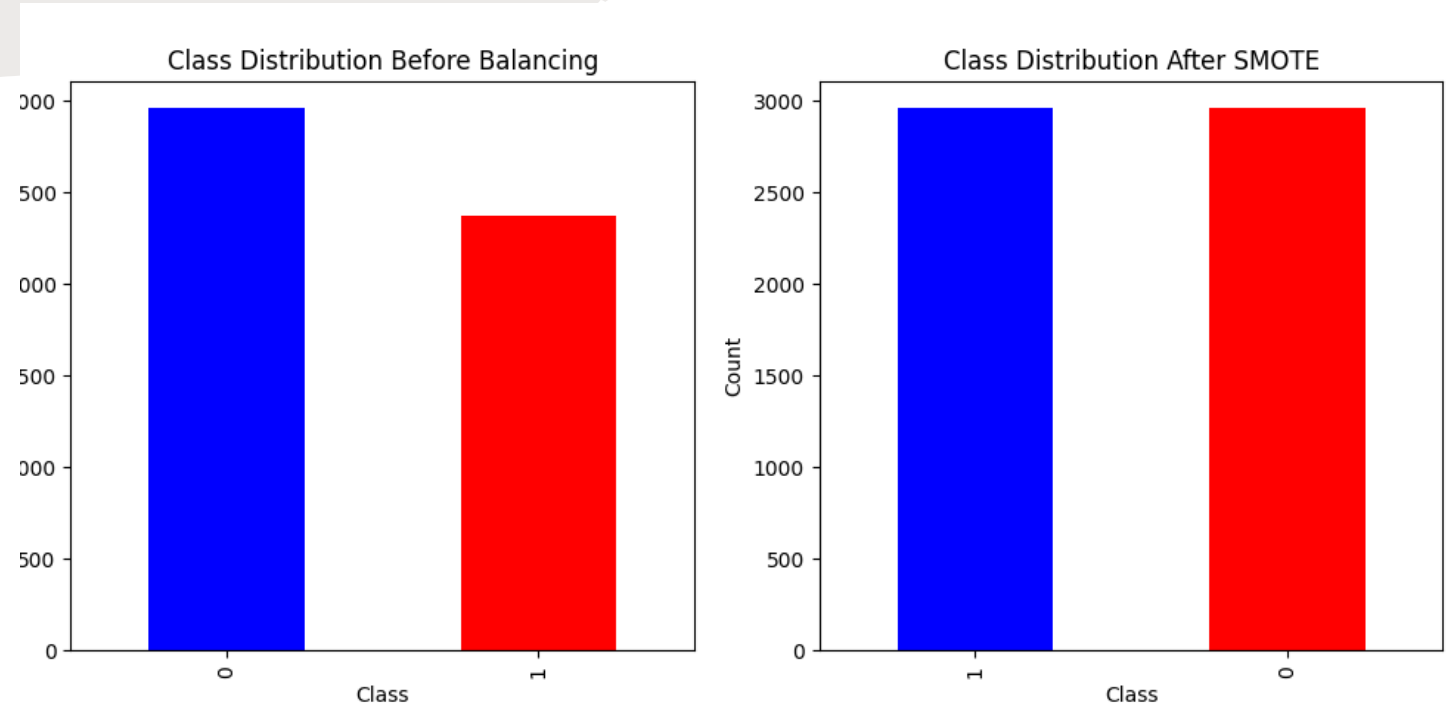
Before SMOTE:

The dataset was imbalanced, with class 0 (2958 instances) being more frequent than class 1 (2374 instances). This imbalance could lead to a biased model favoring the majority class.

After SMOTE:

The dataset was balanced, with both classes having equal counts (2958 instances each). The Synthetic Minority Over-sampling Technique (SMOTE) generated synthetic samples for the minority class, making the dataset more suitable for training machine learning models without bias toward the majority class.

Overall, applying SMOTE ensures fairer model training and improves classification performance, particularly for the minority class.



UNDER-SAMPLING

Under-sampling involves reducing the number of samples in the majority class to balance the dataset. It's important to do this after balancing to avoid overfitting and ensure the model learns from diverse data.

Under-sampling helps reduce the total dataset size, ensuring the model learns general patterns instead of just memorizing data. Keeping the dataset too large (after oversampling) might slow down training.

That's why we use Under-sampling helps maintain an optimal balance between classes while improving model efficiency.

We Use Both SMOTE and Under-sampling?

- Highly imbalanced datasets (e.g., fraud detection, rare disease classification).
- When oversampling alone leads to excessive synthetic data.
- To optimize model training speed by reducing dataset size.

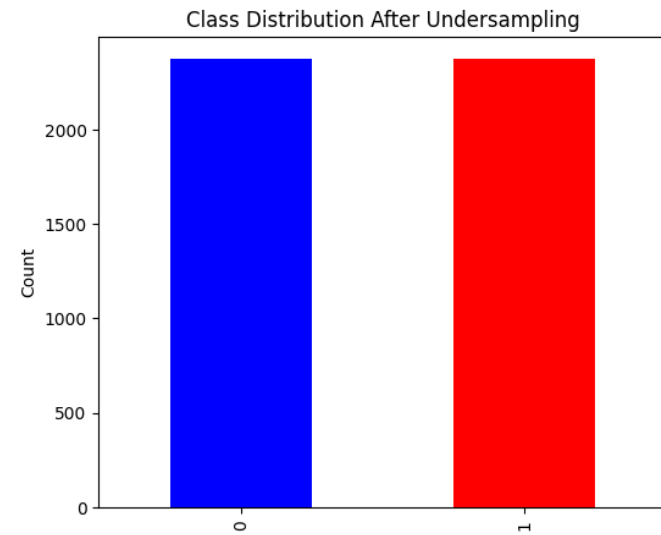
Under-sampling was applied to balance the dataset by reducing the majority class (0) count from 2958 to match the minority class (1) count of 2374.

Before Under-sampling:

The dataset was imbalanced, favoring class 0.

After Under-sampling:

Both classes now have equal representation (2374 each), ensuring that the model does not become biased toward the majority class.



```
# Random Undersampling
under_sampler = RandomUnderSampler(random_state=42)
X_train_under, y_train_under = under_sampler.fit_resample(X_train, y_train)

# Print class distribution after undersampling
print("\nClass distribution after Undersampling:")
print(pd.Series(y_train_under).value_counts())

# Visualize class distribution after undersampling
plt.figure(figsize=(6, 5))
pd.Series(y_train_under).value_counts().plot(kind='bar', color=['blue', 'red'])
plt.title("\nClass Distribution After Undersampling")
plt.xlabel("Class")
plt.ylabel("Count")
plt.show()
```

```
Class distribution after Undersampling:
0    2374
1    2374
Name: count, dtype: int64
```

HYPOTHESIS TESTING

Hypothesis testing is a statistical method used to determine if there is enough evidence to reject a null hypothesis. It helps validate assumptions and draw conclusions from data.

In statistical data analysis, hypothesis testing is used to:

- Validate Assumptions:

Determine if there is enough evidence to support a hypothesis.

- Draw Conclusions:

Make informed decisions based on statistical evidence.

Hypothesis Testing

```
# Hypothesis 1: Checking if mean of Open and Close prices are significantly different
# Reason: We want to determine if there is a statistically significant difference between Open and Close prices, which would indicate potential market trends.
stat1, p_value1 = stats.ttest_ind(df['Open'], df['Close'], nan_policy='omit')
print("Hypothesis 1 Testing Results:")
print(f"T-statistic: {stat1:.4f}, P-value: {p_value1:.4f}")
if p_value1 < 0.05:
    print("Reject the null hypothesis: There is a significant difference between Open and Close prices.")
else:
    print("Fail to reject the null hypothesis: No significant difference between Open and Close prices.")
```

Hypothesis 1 Testing Results:

T-statistic: 0.0024, P-value: 0.9980

Fail to reject the null hypothesis: No significant difference between Open and Close prices.

```
# Hypothesis 2: Checking if mean of High and Low prices are significantly different
# Reason: This test helps us understand whether the variation between High and Low prices is statistically significant, which could indicate market volatility.
stat2, p_value2 = stats.ttest_ind(df['High'], df['Low'], nan_policy='omit')
print("Hypothesis 2 Testing Results:")
print(f"T-statistic: {stat2:.4f}, P-value: {p_value2:.4f}")
if p_value2 < 0.05:
    print("Reject the null hypothesis: There is a significant difference between High and Low prices.")
else:
    print("Fail to reject the null hypothesis: No significant difference between High and Low prices.")
```

Hypothesis 2 Testing Results:

T-statistic: 1.6334, P-value: 0.1024

Fail to reject the null hypothesis: No significant difference between High and Low prices.

• Cryptocurrency Price Analysis •

MODEL TRAINING

Model training in statistical analysis refers to the process of teaching a mathematical model to recognize patterns in data.

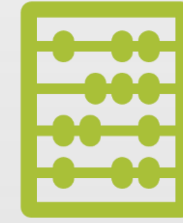
This involves using a dataset to adjust the model's parameters so it can make accurate predictions or uncover insights. During the model training phase, the model is exposed to the data and learns from it.

This process involves using an algorithm (e.g., Linear Regression, Decision Trees, Neural Networks) to identify relationships between input features and the target variable.

For supervised learning, this involves minimizing the error between the predicted output and the actual target value by adjusting the model's internal parameters.

For unsupervised learning, the model tries to identify patterns or groupings in the data without explicit target labels. The goal of model training is to find the optimal set of parameters that result in the best predictive performance.

Logistic Regression, Decision Trees, and Random Forests are three popular algorithms used in machine learning, particularly for classification tasks



Logistic
Regression

Decision Tree



Random
Forest

• Cryptocurrency Price Analysis •

LOGISTIC REGRESSION

Logistic regression is a supervised machine learning algorithm that accomplishes binary classification tasks by predicting the probability of an outcome, event, or observation.

Advantages:

- Simple and easy to implement.
- Efficient for binary classification problems.
- Provides probabilities, making it useful for applications where the probability of a prediction is important.

Logistic Regression



DECISION TREE

A decision tree is a non-parametric supervised learning algorithm, which is utilized for both classification and regression tasks.

Advantages:

- Easy to understand and interpret, as they can be visualized.
- Can handle both numerical and categorical data.
- No need for feature scaling (like normalization).



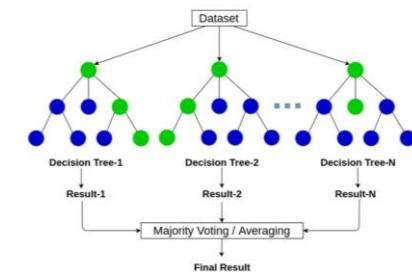
RANDOM FOREST

Random forests or random decision forests is an ensemble learning method for classification, regression and other tasks that works by creating a multitude of decision trees during training.

Advantages:

- High accuracy due to the ensemble of trees.
- Can handle large datasets with higher dimensionality.
- More robust than a single decision tree and less prone to overfitting.
- Can handle both classification and regression tasks.

Random Forest



```
# Model Training
```

```
# Model Training with Cross Validation
```

```
models = {  
    "Logistic Regression": LogisticRegression(),  
    "Decision Tree": DecisionTreeClassifier(),  
    "Random Forest": RandomForestClassifier()  
}
```

```
for name, model in models.items():  
    scores = cross_val_score(model, X_train_smote, y_train_smote, cv=5, scoring='accuracy')  
    print(f"\n{name} Cross-Validation Accuracy: {scores.mean():.4f} (+/- {scores.std():.4f})\n")
```

```
    model.fit(X_train_smote, y_train_smote)  
    y_pred = model.predict(X_test)
```

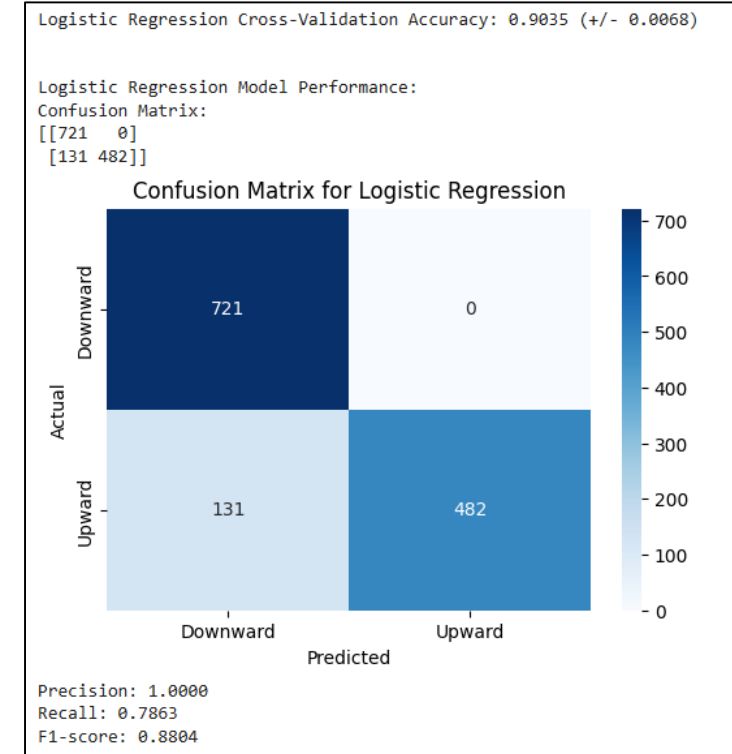
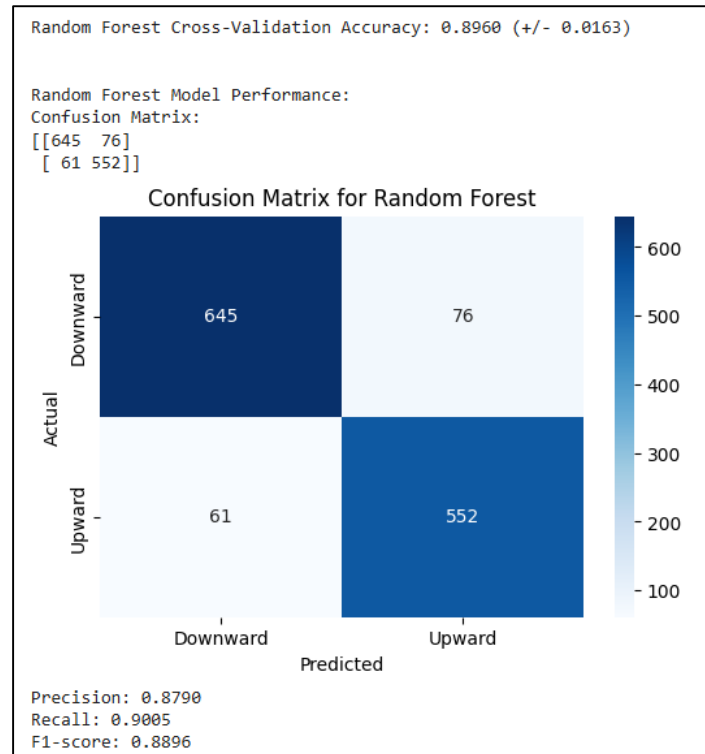
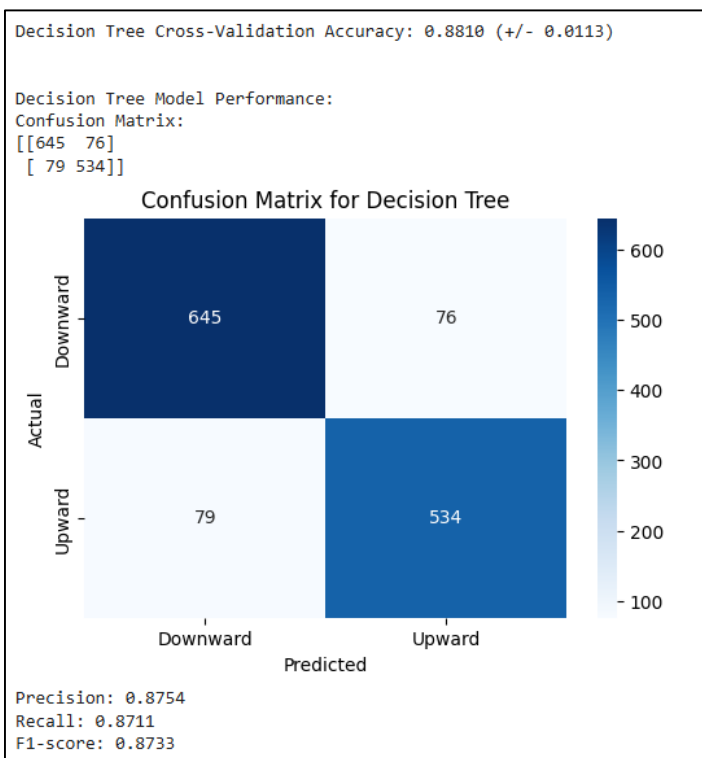
```
    print(f"\n{name} Model Performance:")  
    print("Confusion Matrix:")  
    cm = confusion_matrix(y_test, y_pred)  
    print(cm)
```

```
# Confusion Matrix Visualization
```

```
plt.figure(figsize=(6, 4))  
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Downward', 'Upward'], yticklabels=['Downward', 'Upward'])  
plt.xlabel("Predicted")  
plt.ylabel("Actual")  
plt.title(f"Confusion Matrix for {name}")  
plt.show()
```

```
# Classification Report
```

```
report = classification_report(y_test, y_pred, output_dict=True)  
print(f"Precision: {report['1']['precision']:.4f}")  
print(f"Recall: {report['1']['recall']:.4f}")  
print(f"F1-score: {report['1']['f1-score']:.4f}\n")
```



Logistic Regression is ideal when you need a simple, interpretable model for binary classification and when the relationship between the features and the target is assumed to be linear.

Decision Trees are better for non-linear data and situations where interpretability is important. They may overfit, so controlling tree depth is crucial.

Random Forests provide a powerful and accurate ensemble model, reducing the risk of overfitting and improving generalization by aggregating predictions from multiple trees, making them well-suited for complex and high-dimensional data.

```
# Dictionary to store model performance
```

```
model_performance = {}
```

```
for name, model in models.items():
```

```
    scores = cross_val_score(model, X_train_smote, y_train_smote, cv=5, scoring='accuracy')
```

```
    model.fit(X_train_smote, y_train_smote)
```

```
    y_pred = model.predict(X_test)
```

```
    report = classification_report(y_test, y_pred, output_dict=True)
```

```
# Store accuracy and F1-score for class '1'
```

```
model_performance[name] = {
```

```
    "Accuracy": scores.mean(),
```

```
    "F1-score": report['1']['f1-score']
```

```
}
```

```
# Convert to DataFrame for better visualization
```

```
performance_df = pd.DataFrame(model_performance).T
```

```
# Find the best model based on F1-score
```

```
best_model = performance_df["F1-score"].idxmax()
```

```
print("\nModel Performance Comparison:")
```

```
print(performance_df)
```

```
print(f"\nBest Model: {best_model} (Highest F1-score: {performance_df.loc[best_model, 'F1-score']:.4f})")
```

Model Performance Comparison:

	Accuracy	F1-score
Logistic Regression	0.903482	0.880365
Decision Tree	0.882353	0.864600
Random Forest	0.897058	0.887813

Best Model: Random Forest (Highest F1-score: 0.8878)

Can we predict future cryptocurrency prices using historical data?

Our models suggest future price predictions are possible.

Regression models (Linear, Ridge, Lasso) struggled with non-linearity, indicating crypto prices don't follow simple patterns.

Classification models (Random Forest, Decision Tree, Logistic Regression) performed better in predicting price trends rather than exact values. The Random Forest model provided the best results, showing the importance of ensemble methods in crypto price trend prediction.

- Cryptocurrency Price Analysis
 - How Our Model Helps in Crypto Analysis ?
- Random Forest emerged as the best classification model due to its superior precision-recall balance. Crypto price movements are not entirely random—they exhibit patterns influenced by volume, market cap, and external factors. Classification models are more effective than regression models for predicting crypto price trends. By using classification-based predictions, traders and investors can develop risk-mitigating strategies for better decision-making.



STRENGTH

- › High Accuracy Models: Linear Regression (90.35%) and Random Forest (strong recall and F1-score), show strong predictive performance.
- › Strong Precision (Linear Regression): The linear regression model has perfect precision (1.0000), meaning no false upward predictions, which is critical in volatile markets like crypto.
- › Balanced Performance (Random Forest): Random Forest shows the most balanced metrics (precision, recall, and F1-score), making it reliable for practical use.
- › Good Detection of Upward Trends: Both Decision Tree and Random Forest models have good recall scores (>86%), helping identify potential upward trends.

OPPORTUNITIES

- › Advanced Modeling: Introduce ensemble methods (e.g., Gradient Boosting, XGBoost) or deep learning models (e.g., LSTM, GRU) to capture temporal patterns in crypto prices better.
- › Deployment Potential: With strong model performance, there's a clear opportunity to deploy the model as a trading tool or dashboard.
- › Automated Trading: The results can be integrated into an automated crypto trading strategy using APTs from exchanges like Binance or Coinbase.

WEAKNESS

- › Linear Regression Recall Limitation: Linear Regression, despite high precision, misses 21.37% of upward movements (recall of 0.7863), which might limit profitability.
- › Basic Models Only: The project sticks to classic models; exploring advanced models (e.g., XGBoost, LSTM for time series) could enhance predictive power.

THREAT

- › Crypto Market Volatility: The cryptocurrency market is highly volatile and can change due to factors not captured in historical price data (e.g., regulations, hacks).
- › Overfitting Risk (Tree-Based Models): The Decision Tree and Random Forest may still suffer from overfitting on unseen data if not carefully tuned.
- › Data Drift: Cryptocurrency markets evolve quickly: models trained on past data might

FUTURE RESEARCH DIRECTION

- **Deep Learning Integration:**
Neural networks (LSTMs, transformers) could further improve prediction accuracy. Incorporating real-time news sentiment and on-chain data could refine price movement predictions.
- **Enhancing Model Interpretability:**
Applying SHAP (SHapley Additive exPlanations) values to understand which features contribute most to predictions.
- **Blockchain Data Utilization:**
Using on-chain metrics (e.g., transaction volume, wallet activity) alongside historical prices for better insights.
- **Multimodal Data Fusion:**
Combining price trends, order book data, and global economic indicators to enhance classification models.





Thank You